

Project Executable Files

Date	15 March 2026
TeamID	xxxxxx
Project Name	Personalized Networking Assistant
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	No(Local Deployment)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

Personalized-Networking-Assistant/

```

— app.py                # Main Streamlit Dashboard (UI, Live Metrics, Tabs, Feedback Logger)
— event_analyzer.py     # Theme & Topic Extraction module (DistilBERT emulation / Gemini)
— topic_generator.py    # Conversation Prompt Generation module (GPT-2 emulation / Gemini)
— fact_checker.py       # Fact Verification Service (Wikipedia API Python wrapper integration)
— test_services.py      # Complete suite of unit tests for core modules (pytest)
— history.json          # Persistent Local Log file containing past analyzed events
— feedback.json         # Persistent Local Log file containing thumbs up/down logs
— package.json          # Applet system configuration mapping dev/start scripts to Streamlit
— metadata.json         # Application metadata and service identification
— .env.example          # Documented runtime environment secrets configuration template
— .gitignore            # Excluded caches, local states, and node dependency artifacts
  
```

Main Project Files

- **app/main.py** – FastAPI backend server
- **frontend/app.py** – Streamlit user interface
- **requirements.txt** – Python dependencies
- **README.md** – Setup and execution guide
- **.env.example** – Environment variable template

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Local Deployment
Login Credentials (Demo)	Not Required
Platform / Hosting Provider	Local Machine (FastAPI + Streamlit)
Repository Link	Project Repository
DemoVideoLink	Project Demonstration Video

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Prerequisites

- Python 3.10 or above
- pip package manager
- Internet connection (for Wikipedia API)
- Virtual environment (recommended)

Step 1: Clone the Repository

```
git clone <repository-url>
```

```
cd Personalized-Networking-Assistant
```

Step 2: Create Virtual Environment

Windows

```
python -m venv venv  
venv\Scripts\activate
```

Linux / macOS

```
python3 -m venv venv  
source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

Create a .env file using the provided .env.example.

Example:

```
OPENAI_API_KEY=your_api_key
```

Step 5: Start the FastAPI Backend

```
uvicorn app.main:app --reload
```

Backend will start at:

```
http://127.0.0.1:8000
```

API documentation:

```
http://127.0.0.1:8000/docs
```

Step 6: Start the Streamlit Frontend

Open a second terminal.

```
streamlit run frontend/app.py
```

The Streamlit interface opens automatically at:

<http://localhost:8501>

Step 7: Using the Application

1. Open the Streamlit application.
2. Enter networking or event details.
3. DistilBERT extracts the event theme.
4. GPT-2 generates personalized conversation starters.
5. Wikipedia API validates factual information.
6. User feedback (👍 / 👎) is stored in JSON files.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround/ Status
1	Wikipedia API requires an active internet connection.	Ensure stable internet connectivity.
2	GPT-2 responses may vary due to model generation randomness.	Retry generation for alternate suggestions.
3	GPT-2 responses may vary due to model generation randomness.	Suitable for prototype; can be replaced with a database in future versions.